

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №2
по дисциплине
«Администрирование и программирование распределенных приложений»

Выполнила:

ст. гр. 320604
К. А. Хаджинова

Проверила:

Ю. О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ	3
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	4
3 ВЫПОЛНЕНИЕ РАБОТЫ	5
3.1 Подготовка среды и запуск	5
3.2 Индивидуальное задание	5
3.3 Результаты работы	8
4 КОНТРОЛЬНЫЕ ВОПРОСЫ	13
4.1 Назначение протоколов <i>TCP</i> и <i>UDP</i> и их отличие	13
4.2 Объекты и методы отправки данных по сети	13
4.3 Пересылка данных в обратном направлении (от клиента к серверу)....	13
4.4 Как определить сетевой адрес своего компьютера	13
ЗАКЛЮЧЕНИЕ	14

1 ЦЕЛЬ РАБОТЫ

Познакомиться с техникой использования сокетных классов *Java*.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Сокет (*socket*) – программная точка соединения двух приложений. Для *TCP*-соединения нужны *IP*-адрес компьютера, номер порта, серверный и клиентский сокет. Протокол *TCP* устанавливает соединение, гарантирует доставку данных, сохраняет их порядок и повторно передает потерянные пакеты. Протокол *UDP* работает без постоянного соединения и не гарантирует доставку.

В приложении «клиент-сервер» параллельно работают два потока: поток-сервер и поток-клиент. Сервер создает *ServerSocket* на заданном порту, метод *accept()* блокирует поток до подключения клиента и возвращает объект *Socket*. Клиент создает *Socket*("127.0.0.1", порт), где 127.0.0.1 – адрес локального компьютера. Сервер отправляет текст через *PrintStream* (методы *println* и *flush*), клиент читает строки из входного потока *DataInputStream* методом *readLine()*. Каждый поток реализуется классом, наследующим *Thread*, и запускается методом *start()*. Графический интерфейс приложения строится на классах *AWT*.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Подготовка среды и запуск

Приложение написано на *Java* и скомпилировано *JDK 17 (Zulu 17.0.15)* командой `javac -encoding UTF-8 -d out src\serv.java`. Запуск: `java -Dfile.encoding=UTF-8 -cp out serv`. Методические указания используют устаревшие методы *AWT* и `DataStream.readLine()`, при компиляции они выдают предупреждения об устаревших методах, но работают.

3.2 Индивидуальное задание

Вариант 2: создать два клиента через потоки. Один клиент добавляет и снимает деньги со счета, второй только снимает. Общее задание: приложение для двух клиентов, работающих с одним счетом, каждому клиенту соответствует свой поток; приложение клиент-сервер с визуальным интерфейсом запускает клиента по нажатию кнопки и отражает состояние счета в текстовом поле.

Реализация: серверный поток *Account* хранит общий счет `static int amount = 200` и слушает порт 3001. Клиентский поток *clientThread* подключается к серверу по адресу 127.0.0.1, отправляет запрос *ADD* или *WITHDRAW*, читает ответ вида *Account:<сумма>* и передает его окну. Размер операции генерируется на сервере случайно: `(int)(Math.random() * 1000)`. Окно *serv* содержит три кнопки: «Клиент 1: добавить», «Клиент 1: снять» (клиент 1 добавляет и снимает) и «Клиент 2: снять» (клиент 2 только снимает), текстовое поле счета и журнал операций. Нажатие кнопки запускает новый поток клиента.

Листинг 3.1 – Серверный поток *Account*

// Серверный поток: хранит счёт и обрабатывает запросы клиентов

```
class Account extends Thread {
    ServerSocket server;
    String amountstring;
    static int amount = 200;

    public void run() {
        try {
            server = new ServerSocket(3001); // Номер сокета
        } catch (Exception e) {
            System.out.println("Ошибка соединения+" + e);
        }
        while (true) {
            Socket s = null;
            try {
                s = server.accept(); // ожидание соединения с клиентом
```

```

    } catch (Exception e) {
        System.out.println("Ошибка" + e);
    }
    try {
        // Чтение запроса клиента (ADD или WITHDRAW)
        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
        String request = in.readLine();

        PrintStream ps = new PrintStream(s.getOutputStream()); // PrintStream
предназначен для текстового вывода
        int amountcur = ((int) (Math.random() * 1000));
        if (request.equals("ADD"))
            amount += amountcur;
        else
            amount -= amountcur;
        amountstring = Integer.toString(amount);
        ps.println("Account:" + amountstring); // передача строки клиенту
        ps.flush();
        s.close(); // Сокетное соединение закрывается
    } catch (Exception e) {
        System.out.println("Ошибка " + e);
    }
}
}
}
}

```

Сервер последовательно принимает подключения, читает строку запроса через *BufferedReader* (пересылка данных от клиента к серверу), изменяет счет и отправляет ответ клиенту. Код клиентского потока приведен в листинге.

Листинг 3.2 – Клиентский поток *clientThread*

// Клиентский поток: отправляет запрос серверу и читает ответ

```

class clientThread extends Thread {
    DataInputStream dis = null;
    Socket s = null;
    String name;
    String request;
    serv window;

    public clientThread(String name, String request, serv window) {
        this.name = name;
        this.request = request;
        this.window = window;
    }
    try {
        s = new Socket("127.0.0.1", 3001);
        dis = new DataInputStream(s.getInputStream());
    } catch (Exception e) {
        System.out.println("Ошибка: " + e);
    }
}

```

```

    }

    public void run() {
        try {
            // Отправка запроса серверу
            PrintStream ps = new PrintStream(s.getOutputStream());
            ps.println(request);
            ps.flush();

            String msg = dis.readLine(); // Клиент читает строку из сокета
            if (msg != null) {
                System.out.println(name + ": " + msg);
                window.showMessageDialog(name + ": " + msg);
            }
            s.close();
        } catch (Exception e) {
            System.out.println("ERRORR+" + e);
        }
    }
}
}

```

Окно приложения и запуск клиентов по кнопкам реализованы классом *serv*, код которого приведен в листинге.

Листинг 3.3 – Окно приложения *serv*

```

// Окно приложения
public class serv extends Frame implements ActionListener {
    Button client1Add = new Button("Клиент 1: добавить");
    Button client1Withdraw = new Button("Клиент 1: снять");
    Button client2Withdraw = new Button("Клиент 2: снять");
    TextField accountField = new TextField("200", 10);
    TextArea log = new TextArea(8, 40);

    public serv() {
        super("Счѐм");
        setLayout(new FlowLayout());
        add(client1Add);
        add(client1Withdraw);
        add(client2Withdraw);
        add(new Label("Счѐм:"));
        add(accountField);
        add(log);

        client1Add.addActionListener(this);
        client1Withdraw.addActionListener(this);
        client2Withdraw.addActionListener(this);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0); // Закрѐть приложение
            }
        })
    }
}

```

```

    });
}

// Запуск клиента по нажатию кнопки
public void actionPerformed(ActionEvent ae) {
    if (ae.getSource() == client1Add)
        new clientThread("Клиент 1", "ADD", this).start();
    else if (ae.getSource() == client1Withdraw)
        new clientThread("Клиент 1", "WITHDRAW", this).start();
    else if (ae.getSource() == client2Withdraw)
        new clientThread("Клиент 2", "WITHDRAW", this).start();
}

// Отображение состояния счёта
public void showMessage(String message) {
    accountField.setText(message.substring(message.indexOf(':', message.indexOf(':') + 1) +
1));
    log.append(message + "\n");
}

public static void main(String args[]) {
    serv f = new serv();
    f.setSize(400, 400);
    f.setVisible(true);
    new Account().start();
}
}

```

3.3 Результаты работы

Окно после запуска показано на рисунке 3.1.

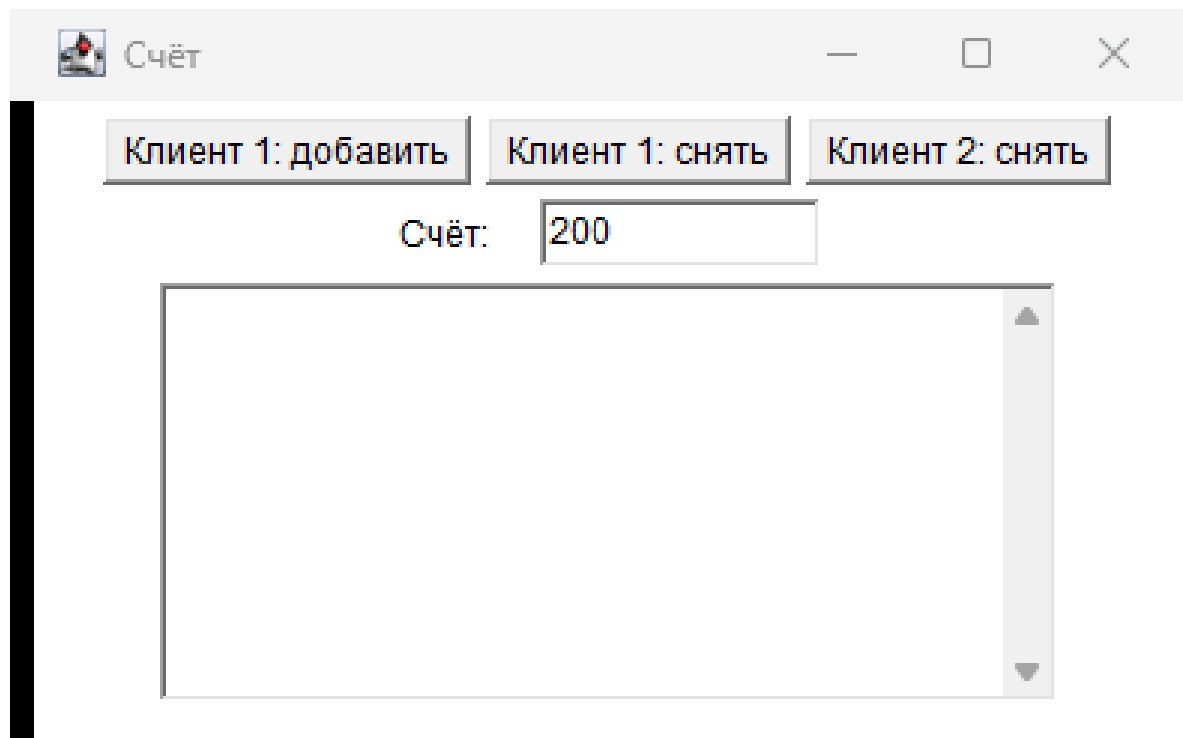


Рисунок 3.1 – Окно приложения после запуска

В поле «Счет» показано начальное значение 200, журнал операций пуст.

Нажатие кнопки «Клиент 1: добавить» запускает поток первого клиента, результат показан на рисунке 3.2.

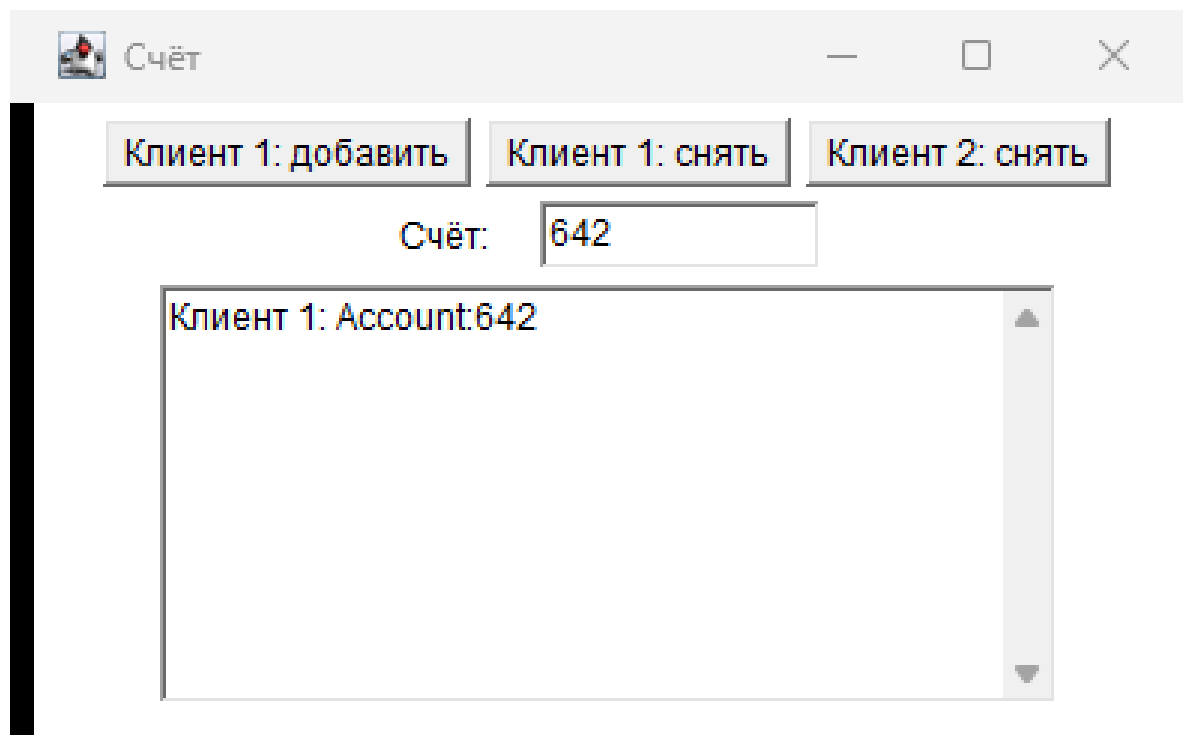


Рисунок 3.2 – Клиент 1 добавил деньги на счет

Сервер прибавил случайную сумму, счет стал равным 642, ответ клиента добавлен в журнал.

Операция снятия клиентом 1 показана на рисунке 3.3.

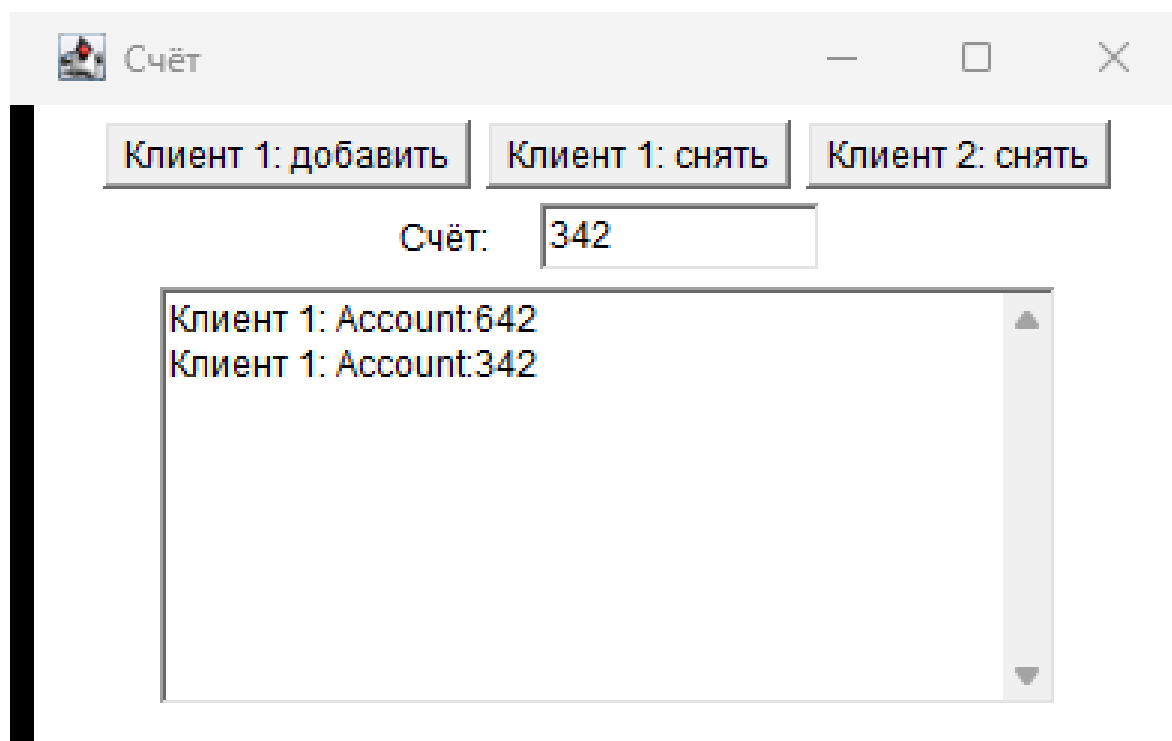


Рисунок 3.3 – Клиент 1 снял деньги со счета

После снятия счет уменьшился с 642 до 342.

Клиент 2 выполняет только снятие, результат показан на рисунке 3.4.

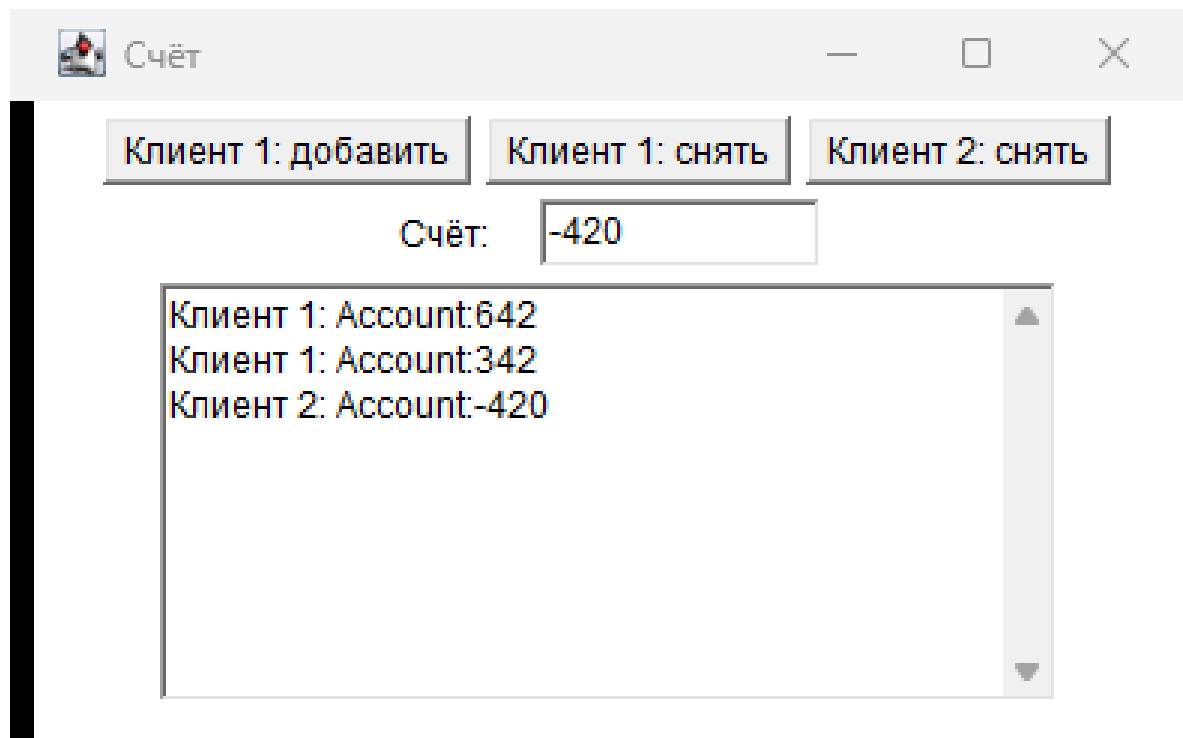


Рисунок 3.4 – Клиент 2 снял деньги со счета

Счет стал равным минус 420: размер операции генерируется случайно, а проверка остатка в задании не предусмотрена. Оба клиента работают с одним общим счетом, в журнале видны операции обоих.

Итоговое состояние после нескольких операций показано на рисунке 3.5.

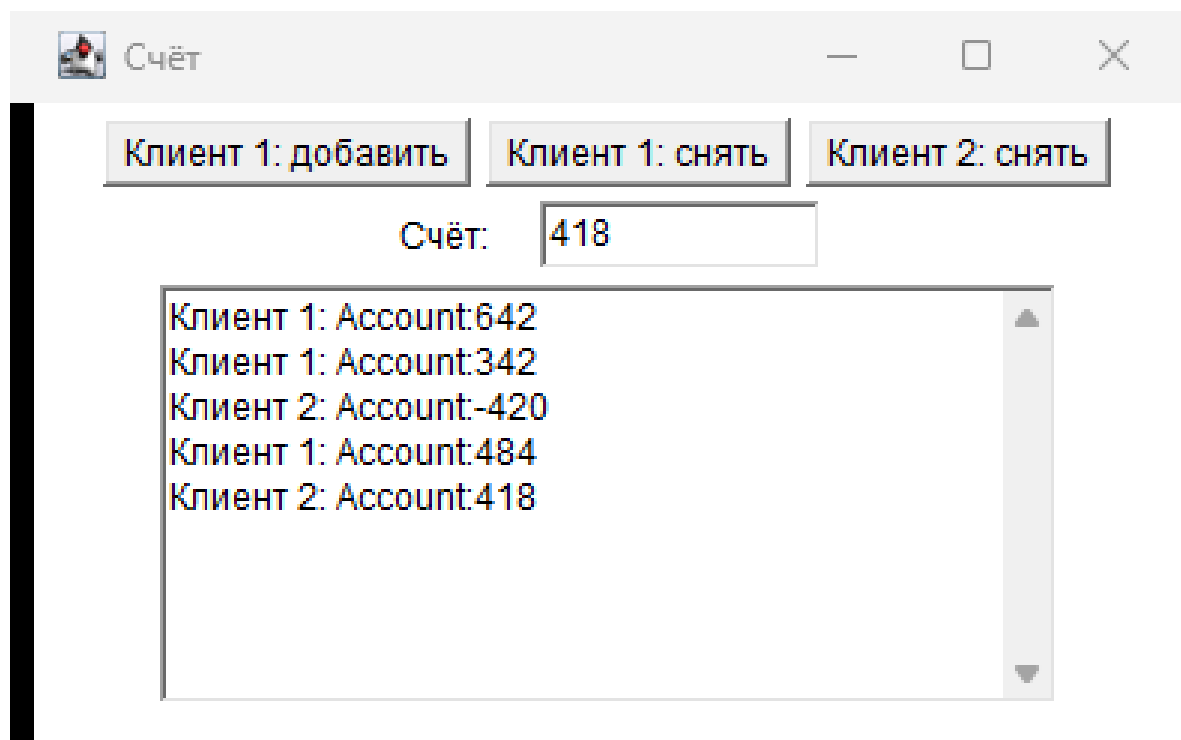


Рисунок 3.5 – Состояние счета после нескольких операций

Последовательность операций клиента 1 (добавление) и клиента 2 (снятие) привела к значению 418. Журнал отражает все пять выполненных операций по порядку.

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Назначение протоколов *TCP* и *UDP* и их отличие

TCP – протокол с установлением соединения: он гарантирует доставку данных, сохраняет порядок пакетов и повторно передает потерянные. *UDP* работает без постоянного соединения, не гарантирует доставку и порядок, но имеет меньшие накладные расходы. В работе используется *TCP*, так как применяются классы *ServerSocket* и *Socket*.

4.2 Объекты и методы отправки данных по сети

Данные отправляются через выходной поток сокета: *s.getOutputStream()* оборачивается в *PrintStream*, методом *println()* записывается строка, методом *flush()* буфер принудительно отправляется. Читаются данные из *s.getInputStream()* через *DataInputStream* или *BufferedReader* методом *readLine()*.

4.3 Пересылка данных в обратном направлении (от клиента к серверу)

Клиент использует *PrintStream* поверх *s.getOutputStream()* и отправляет строку запроса, сервер читает ее через *BufferedReader* поверх *s.getInputStream()* методом *readLine()*. В работе так передаются запросы *ADD* и *WITHDRAW*.

4.4 Как определить сетевой адрес своего компьютера

В командной строке *Windows* выполняется команда *ipconfig*, в *Java* адрес возвращает *InetAddress.getLocalHost().getHostAddress()*. Адрес 127.0.0.1 – *loopback*-адрес, обозначающий этот же компьютер.

ЗАКЛЮЧЕНИЕ

Изучены сокетные классы *Java*: создан серверный поток на *ServerSocket* и клиентские потоки на *Socket*, реализована передача запросов от клиентов к серверу и ответов в обратном направлении. По варианту 2 два клиента работают с общим счетом: клиент 1 добавляет и снимает деньги, клиент 2 только снимает. Приложение с графическим интерфейсом запускает клиента по нажатию кнопки и отражает состояние счета в текстовом поле. Цель работы достигнута.